Q-BIT: BASIC QUANTUM GATE VISUALIZER

ABSTRACT

The "Q-Bit" project represents a sophisticated intersection of quantum information theory and modern full-stack web engineering. As quantum computing transitions from theoretical physics into practical computer science, there is a critical need for accessible, high-fidelity simulation tools that can demystify complex concepts such as superposition, phase interference, and entanglement for the next generation of engineers. This project addresses this gap by providing a comprehensive, 2-qubit quantum gate visualizer built on a robust distributed architecture.

The system architecture is bifurcated into a high-performance **Java 25** backend, powered by the **Spring Boot 4.0** framework, and a highly responsive **React.js** frontend. The backend serves as the primary computational engine, utilizing custom-built complex number libraries and matrix transformation algorithms to simulate the physical behavior of quantum gates like the **Hadamard (H)**, **Pauli-X/Y/Z**, and **Controlled-NOT (CNOT)**. Every user interaction in the frontend triggers a synchronized statevector calculation in the backend, ensuring that the mathematical output—rendered in professional **Dirac notation**—is always physically accurate and reflects real-time circuit changes.

A significant focus of this research was the implementation of a secure, persistent user environment. By integrating a **MySQL database** with **Spring Data JPA**, the application provides a seamless registration and authentication module. This allows for a personalized workspace where user credentials and session states are managed through a secure "Profile" interface, featuring a modern, non-intrusive UI. The frontend utilizes **Tailwind CSS** and **@dnd-kit** to create a dark-themed, "workspace-first" user experience, optimized for high-resolution displays without requiring vertical scrolling.

The final deliverable is an educational tool that effectively translates abstract 4X4 unitary matrices into a visual, drag-and-drop narrative. "Quantum-Sim" successfully demonstrates that by offloading heavy mathematical lift to a compiled Java environment while maintaining a fluid React state, developers can create powerful pedagogical tools that remain accessible to students worldwide. This project lays the groundwork for more advanced multi-qubit simulations and serves as a verified proof-of-concept for the integration of quantum simulators into standard web-based learning management system

INTRODUCTION

2.1 BACKGROUND OF QUANTUM COMPUTING

Quantum computing represents a paradigm shift from classical Boolean logic to the principles of quantum mechanics, such as superposition and entanglement. While classical computers process information in bits (0 or 1), a quantum computer utilizes qubits, which can exist in a linear combination of both states simultaneously. This project, "Q-Bit," focuses on the 2-qubit system, which is the foundational building block for understanding multi-qubit interactions and quantum logic gates.

2.2 THE EDUCATIONAL GAP

Despite the potential of quantum algorithms, the mathematical entry barrier remains high for engineering students. Understanding how a **Hadamard (H)** gate creates superposition or how a **CNOT** gate facilitates entanglement requires a firm grasp of complex number matrices and unitary transformations. Currently, many existing simulators are either strictly command-line based or overly academic, lacking the intuitive drag-and-drop experience required for rapid prototyping and learning.

2.3 PROBLEM STATEMENT

The primary challenge addressed in this project is the creation of a seamless, full-stack environment that handles heavy mathematical lifting on a compiled backend while providing a fluid, reactive frontend. Developing a system that can calculate 2-qubit statevectors (4X4 matrix operations) in real-time and return results in **Dirac notation** requires a tightly coupled architecture between **Java 25** and **React.js**. Additionally, the requirement for a persistent user module necessitates a robust database integration with **MySQL**.

2.4 SCOPE AND ORGANIZATION

The scope of this report covers the end-to-end development of the "Quantum-Sim" application. This includes:

- The design of the **MySQL** database schema for user authentication.
- The implementation of the **Spring Data JPA** repository for secure data access.
- The development of the **React** drag-and-drop workspace using **@dnd-kit/core**.
- The mathematical modeling of quantum gates through Java-based complex number.

OBJECTIVES

The development of Q-BIT was guided by a set of core technical and pedagogical objectives designed to create a robust, full-stack quantum simulation environment.

3.1 HIGH-FIDELITY QUANTUM CIRCUIT VISUALIZATION

The primary objective was to design a front-end interface that translates abstract mathematical operations into a tangible, drag-and-drop experience.

- User Interaction: To implement a workspace using React.js and @dnd-kit/core where users can intuitively place gates on parallel qubit wires.
- Visual Feedback: To provide a high-tech "Dark Mode" aesthetic that reduces eye strain and emphasizes the modern nature of quantum technology through Tailwind CSS.
- Single-Screen Accessibility: To engineer the UI layout such that all critical components—the Library, the Wires, and the Results—fit within a single viewport to eliminate the need for vertical scrolling.

3.2 ROBUST FULL-STACK ARCHITECTURE AND DATA PERSISTENCE

A secondary but vital objective was to transition the project from a local script into a scalable web application.

- Authentication and Security: To develop a secure login and registration module that stores user credentials in a MySQL database, ensuring a private session for every user.
- State Persistence: To utilize Spring Data JPA to manage the Object-Relational Mapping (ORM) between the Java User entity and the relational database tables.
- RESTful API Integration: To establish a high-speed communication bridge between the React frontend and the Spring Boot backend using Axios for real-time data exchange.

3.3 MATHEMATICAL ACCURACY IN QUANTUM SIMULATION

The core scientific objective was to ensure that every visual interaction yields a mathematically sound quantum result.

- Unitary Transformation Engine: To build a Java-based backend capable of processing 2X2 matrices for single-qubit gates and 4X4 matrices for interaction gates like CNOT and SWAP.
- Complex Number Arithmetic: To develop a custom ComplexNumber class in Java to

handle the real and imaginary components of probability amplitudes without loss of precision.

- Dirac Notation Rendering: To format the final 4-element statevector into a standard Dirac notation string, making the result immediately readable to anyone familiar with quantum mechanics.

3.4 PERFORMANCE AND REAL-TIME RESPONSIVENESS

To provide a "live" feel to the simulation, the system was optimized for low-latency updates.

- Triggered Re-computation: To implement React useEffect hooks that automatically send the updated circuit state to the backend whenever a gate is moved or a qubit is toggled.
- Server-Side Efficiency: To utilize Spring Boot's rapid request handling to process matrix multiplications and return results in milliseconds, preventing UI lag.

TOOLS AND TECHNIQUES

4.1 FRONTEND DEVELOPMENT ENVIRONMENT

The user interface of Q-BIT is engineered to provide a high-performance, reactive experience that can handle complex drag-and-drop states without performance degradation.

- **React.js (Vite):** Chosen as the core library for its component-based architecture, which allows for the modular development of gates, wires, and the results footer. Vite is utilized as the build tool to provide lightning-fast Hot Module Replacement (HMR) during the development of the quantum design canvas.
- **Tailwind CSS:** A utility-first CSS framework used to implement the "Dark Space" aesthetic. It facilitates the creation of complex layouts, such as the glassmorphism effect on the login container and the neon glows on the qubit toggles, using standard utility classes like backdrop-blur-xl and shadow-cyan-500/20.
- **@dnd-kit/core:** A lightweight, performant drag-and-drop toolkit for React. This library is essential for mapping the "Toolbox" gates to specific coordinates on the "DroppableWire" components, ensuring that gate instances are correctly assigned to either Wire 1 or Wire 2 based on user interaction.
- **Axios:** An isomorphic HTTP client used to facilitate communication between the React frontend and the Spring Boot API. Axios handles the asynchronous POST requests that transmit the current circuit state (gate types and initial values) to the backend for mathematical processing.

4.2 BACKEND ENGINEERING AND COMPUTATION

The backend is developed using the **Spring Boot MVC** framework. This structure allows the application to efficiently map incoming HTTP requests from the React frontend to specific Java methods, facilitating secure authentication and accurate quantum state calculations through a centralized controller logic.

The backend serves as the mathematical heart of the application, transforming visual circuit designs into linear algebraic results.

- **Spring Boot 4.0:** Utilized to create a robust, production-ready REST API. Spring Boot's auto-configuration and embedded server capabilities allow for rapid deployment of the /api/quantum/simulate and /api/auth endpoints.
- **Java 25:** The latest Long-Term Support (LTS) version of Java was selected for its high-

performance execution of matrix operations and modern syntax. Java's strong typing system ensures that complex number arithmetic remains precise throughout the simulation.

- **Maven:** Used as the project management and comprehension tool. Maven handles all external dependencies, including the MySQL connector and Spring Data JPA, through a centralized pom.xml configuration file.
- **Lombok:** A Java library integrated to minimize boilerplate code. By using annotations like @Data, the project maintains clean entity and model classes, automatically generating getters, setters, and constructors required for the user authentication logic.

4.3 DATABASE AND PERSISTENCE LAYER

The application requires a persistent storage layer to manage user accounts and ensure a secure design environment.

- **MySQL Server:** A relational database management system used to store user credentials. MySQL provides the ACID (Atomicity, Consistency, Isolation, Durability) compliance necessary for secure user registration and login.
- **Spring Data JPA / Hibernate:** These technologies act as the Object-Relational Mapping (ORM) layer. JPA allows the backend to interact with MySQL using Java objects instead of manual SQL queries, significantly reducing the risk of SQL injection and improving code maintainability.
- **H2 Database (Development):** An in-memory database used during the initial testing phase to verify authentication logic before migrating to a persistent MySQL instance.

4.4 DEVELOPMENTAL TOOLS

- **Eclipse IDE:** The primary environment for backend development, offering advanced debugging tools for tracing matrix multiplications in the QuantumController.
- **VS Code:** Utilized for frontend development due to its superior support for React, Tailwind CSS IntelliSense, and ESLint for error checking.
- **MySQL Workbench:** Used for designing the qbit database schema and manually verifying that user registration data is correctly stored in the users table.

PROJECT OVERVIEW

The **Q-BIT** system is a sophisticated multi-tier application designed to facilitate real-time quantum circuit simulation. The overview of this project is best understood through its functional architecture, which separates user management, the visual design interface, and the mathematical computation engine.

5.1 SYSTEM ARCHITECTURE AND TOPOLOGY

The project follows the Model-View-Controller (MVC) architectural pattern to ensure a clean separation of concerns. The Model is represented by the MySQL database and JPA entities that manage user data; the View is implemented using React.js to provide an interactive quantum workspace; and the Controller is managed by Spring Boot REST APIs that handle the logic between the user interface and the quantum computational engine.

It also follows a standard **Client-Server Architecture** model, specifically optimized for high-frequency data exchange.

- **The Presentation Layer (Frontend):** Built with **React.js**, this layer manages the state of the quantum wires and the positioning of gates. It utilizes a "Reactive State" model, where any change to the circuit—such as toggling a qubit from $|0\rangle$ to $|1\rangle$ — immediately triggers a re-evaluation of the entire system state.
- **The Application Layer (Backend):** The **Spring Boot** server acts as the "brain" of the project. It hosts the RESTful endpoints that receive circuit data, processes the linear algebra required for quantum simulation, and interfaces with the database for identity management.
- **The Persistence Layer (Database):** A **MySQL** instance stores the structural data of the application, primarily focused on user identity and authentication records.

5.2 THE USER JOURNEY AND IDENTITY MODULE

A core component of the project overview is the security-first approach to the user workspace.

- **Authentication Flow:** Upon visiting the application, the user is presented with a unified Login/Registration portal. This module ensures that only authorized users can access the quantum workspace. The registration process captures a unique username and password, which is then mapped to a **User Entity** in the backend and persisted in the MySQL users table.

- **Session Management:** Once authenticated, the React frontend stores the user's session state, enabling the "Profile" icon in the top-right corner. This icon displays the user's initial and provides a secure "Logout" mechanism, ensuring that the workspace remains private.

5.3 THE QUANTUM DESIGN CANVAS (WORKSPACE)

The heart of the project is the interactive workspace, designed to mimic a laboratory environment.

- **Gate Library:** A sidebar containing a curated set of quantum gates (**H, X, Y, Z, S, T, I, CNOT, SWAP**). Each gate is a draggable component with specific CSS properties and metadata identifying its unitary matrix type.
- **Qubit Wires:** Two parallel horizontal lines representing **Wire 1** and **Wire 2**. These wires are "Droppable" zones. When a gate is dropped onto a wire, the frontend updates an array of objects that represents the chronological sequence of operations for that specific qubit.
- **Real-Time Result Footer:** Located at the bottom of the workspace, this section remains fixed and updates every time the circuit state changes. It displays the final **Statevector** in Dirac notation, providing the user with immediate mathematical feedback.

5.4 DATA FLOW AND COMMUNICATION

The overview of the project is incomplete without describing the "Handshake" between technologies:

1. **Event Capture:** A user drags a "Hadamard" gate onto Wire 1.
2. **State Update:** React updates the wire1 state array.
3. **API Request:** An **Axios** POST request is dispatched to the backend, carrying a JSON payload containing the gate sequence for both wires and the initial qubit values.
4. **Backend Computation:** The QuantumController receives the JSON, maps the strings to matrices, performs the matrix multiplications, and generates a Dirac string.
5. **UI Sync:** The result is sent back to React and displayed in the footer.

WORKING (PROCESS FLOW)

The operational logic of **Q-BIT** is defined by a highly synchronized data pipeline. The process flow is divided into three distinct phases: the **Authentication Phase**, the **Circuit Design Phase**, and the **Mathematical Computation Phase**.

Component	Technology	MVC Role
Database	MySQL / Hibernate	**Model** (Data Persistence)
Backend	Spring Boot	**Controller** (Business Logic)
Frontend	React.js	**View** (User Presentation)

6.1 THE AUTHENTICATION AND INITIALIZATION PHASE

Before the quantum simulation environment is accessible, the system must establish a secure user session.

1. **User Input:** The process begins when a user enters credentials into the React-based login or registration form.
2. **API Dispatch:** React utilizes **Axios** to send a POST request to the /api/auth/login or /api/auth/register endpoint in the Spring Boot backend.
3. **Database Validation:** The AuthController receives the User object and invokes the UserRepository. Spring Data JPA executes a SELECT query on the MySQL users table to verify the username and password.
4. **Session Grant:** If credentials match, the backend returns a **200 OK** status. React then updates the isLoggedIn state to true, which triggers a conditional re-render of the entire UI, replacing the login screen with the Quantum Workspace.

6.2 THE INTERACTIVE CIRCUIT DESIGN PHASE

Once inside the workspace, the process flow shifts to the **DndContext** (Drag and Drop) logic.

1. **Gate Selection:** The user selects a gate (e.g., a **Hadamard** gate) from the Library sidebar. The @dnd-kit/core library tracks the "Active" element.
2. **Collision Detection:** As the user drags the gate over **Wire 1** or **Wire 2**, the system calculates "Droppable" zones.
3. **State Transformation:** Upon releasing the mouse (the onDragEnd event), the

application determines the destination wire. It generates a unique ID for the gate instance (e.g., H-171213456) and appends it to the corresponding state array (wire1 or wire2).

4. **Qubit Initialization:** Simultaneously, the user can click the $|0\rangle$ or $|1\rangle$ buttons to toggle the initial state of the qubits, which updates the init state object in React.

6.3 THE MATHEMATICAL COMPUTATION PHASE

This is the most critical part of the process flow, where the visual circuit is converted into a quantum statevector.

1. **JSON Payload Construction:** Any change to the wires or initial values triggers a useEffect hook in React. This hook gathers the current names of all gates on both wires into a JSON payload:

   { "wire1": ["H", "X"], "wire2": ["CNOT"], "init1": 0, "init2": 0 }

2. **RESTful Transmission:** This payload is sent via a POST request to the /api/quantum/simulate endpoint.
3. **Unitary Matrix Mapping:** In the Java backend, the QuantumController iterates through the gate lists. It uses a factory pattern to map string names (like "H") to their corresponding unitary matrix representations.
4. **Statevector Multiplications:** * The system initializes a 4-dimensional complex vector representing the $|00\rangle$ state.
   - For single-qubit gates, a **Kronecker Product** is used to create a 4X4 operator.
   - For 2-qubit gates (CNOT/SWAP), the specific 4X4 matrix is applied directly.
   - The backend performs sequential matrix-vector multiplications for every gate in the circuit.
5. **Dirac Formatting:** The resulting statevector is converted into a human-readable **Dirac notation** string (e.g., $1.00|00\rangle$).
6. **UI Synchronization:** The final string is sent back to the React frontend and rendered in the Result Footer, completing the cycle.

SKILLS ACQUIRED

The development of **Q-BIT** required a multidisciplinary skill set, blending traditional full-stack engineering with quantum information science and systems optimization.

7.1 FULL-STACK WEB ENGINEERING

The most prominent skill utilized was the integration of a decoupled architecture, requiring proficiency in both client-side and server-side technologies.

- **React.js Component Design:** Mastery of the component lifecycle and state management was essential. Skills were applied in creating reusable UI elements like the **Gate** and **DroppableWire**, and managing complex state objects that track the chronological order of quantum operations.
- **RESTful API Development:** Designing and implementing secure endpoints using **Spring Boot 4.0**. This involved defining clear JSON contracts between the React frontend and Java backend to ensure seamless data transmission for both authentication and simulation.
- **Asynchronous Communication:** Utilizing **Axios** to handle non-blocking HTTP requests, ensuring that the UI remains responsive even while the backend performs heavy matrix multiplications.

7.2 DATABASE MANAGEMENT AND SECURITY

Implementing the user identity module required a transition from basic coding to database administration.

- **Relational Database Design:** Skills were utilized in designing a **MySQL** schema that supports user persistence. This included setting up primary keys, ensuring unique constraints on usernames, and managing data types for encrypted passwords.
- **Object-Relational Mapping (ORM):** Proficient use of **Spring Data JPA** and **Hibernate** to bridge the gap between Java's object-oriented model and MySQL's relational model. This skill eliminated the need for manual SQL boilerplate and improved the security of the data layer.

7.3 QUANTUM MATHEMATICAL MODELING

A unique aspect of this project was the translation of theoretical physics into algorithmic logic.

- **Linear Algebra and Matrix Calculus:** Deep application of matrix-vector multiplication, specifically using 4X4 unitary matrices to represent quantum gates. Understanding the **Kronecker Product** was a vital skill for calculating how single-qubit gates affect a 2-qubit system.
- **Complex Number Arithmetic:** Implementing a custom mathematical engine in Java to handle the real and imaginary parts of probability amplitudes. This required a high level of precision to ensure the final statevector adhered to the normalization constraint (the sum of squares equals 1).

7.4 UI/UX DESIGN AND BRANDING

Designing a professional educational tool required an eye for aesthetics and usability.

- **Tailwind CSS and Responsive Layouts:** Utilizing utility-first CSS to build a "one-page" dashboard. Skills included implementing **Glassmorphism**, managing absolute positioning for the copyright footer, and using backdrop blurs to maintain text readability against a complex background image.
- **User-Centric Design:** Organizing the workspace logically—placing the gate library on the left, the wires in the center, and the results at the bottom—to follow the natural "left-to-right" reading flow of a circuit diagram.

7.5 DEBUGGING AND TROUBLESHOOTING

- **Full-Stack Debugging:** The ability to trace an error from the browser console into the Spring Boot logs and finally into the MySQL database. This was particularly evident when resolving the **MySQL Driver** ClassNotFoundException and fixing the **Lombok** configuration in Eclipse.

CHALLENGES FACED

The development of **Q-BIT** involved navigating complex integrations between disparate technologies. Each layer of the stack presented unique technical hurdles that required iterative debugging and architectural refinement.

8.1 BACKEND CONFIGURATION: THE MYSQL DRIVER CONFLICT

One of the most critical early-stage challenges was the failure of the Spring Boot ApplicationContext due to a missing database driver.

- **The Symptom:** The application crashed immediately upon startup with a ClassNotFoundException: com.mysql.cj.jdbc.Driver.
- **Root Cause Analysis:** While the application.properties file was correctly configured with the MySQL URL and credentials, the project's build path did not actually contain the physical .jar file required to communicate with a MySQL server.
- **Resolution:** The Maven pom.xml was updated to include the mysql-connector-j dependency with a runtime scope. This allowed Maven to automatically download and inject the driver into the classpath, successfully resolving the bean creation error for the dataSource.

8.2 IDE INTEGRATION: LOMBOK AND ECLIPSE COMPILATION

During the development of the **User Entity**, a disconnect occurred between the source code and the compiled bytecode.

- **The Symptom:** Eclipse flagged errors in the AuthController, claiming that methods like getUsername() and getPassword() were undefined for the User class, despite the presence of the @Data annotation.
- **Root Cause Analysis:** The **Lombok** library generates these methods at compile-time. However, the Eclipse IDE requires a specific "Agent" to be installed in its core files (eclipse.ini) to recognize these generated methods during the editing phase.
- **Resolution:** The **Lombok.jar** was manually executed as a standalone installer to patch the Eclipse IDE. After a full IDE restart and a "Project Clean," the red error markers vanished, confirming that the IDE could now "see" the dynamically generated getters and setters.

8.3 UI DESIGN: MANAGING VERTICAL REAL ESTATE

A significant design challenge was maintaining a "No-Scroll" policy while adding more informational components to the workspace.

- **The Symptom:** After adding the 8-line **Project Concept** section and the **Copyright Footer**, the "Results Footer" was pushed off the bottom of the screen, forcing the user to scroll to see the quantum output.
- **Root Cause Analysis:** The default block-level element flow and standard Tailwind margins (mb-12, p-10) were consuming too many vertical pixels for a standard 1080p display.
- **Resolution:** The UI was re-engineered using **Absolute Positioning** for the copyright text and a reduction in vertical "gaps" (changing gap-16 to gap-10). This ensured that the header, concept, wires, and results all stayed within the single "Viewport Height" (100vh), creating a professional dashboard feel.

8.4 FRONTEND LOGIC: DRAG-AND-DROP STATE INTEGRITY

Ensuring that gates remained locked to their respective wires during high-frequency updates was a complex state management task.

- **The Symptom:** Dragging a gate would occasionally cause it to disappear or "jump" between Wire 1 and Wire 2 incorrectly.
- **Root Cause Analysis:** The onDragEnd handler was initially struggling to distinguish between the "Toolbox" source and the "Wire" destination, leading to ID collisions.
- **Resolution:** A robust ID naming convention was implemented (e.g., ${type}-${Date.now()}) to ensure every gate instance had a globally unique identifier. Conditional logic was added to the handler to strictly filter gates into the wire1 or wire2 state arrays based on the over.id property.

CONCLUSION

9.1 PROJECT SYNTHESIS

The development of **Q-BIT** has successfully demonstrated that modern web technologies—specifically the **Spring Boot** and **React.js** ecosystem—are highly capable of hosting complex scientific simulations. By offloading the linear algebraic computations to a compiled **Java 25** environment, the application maintains mathematical rigor while providing a fluid, high-fidelity user interface. The project met all its core objectives, including secure user authentication via **MySQL**, real-time matrix-vector transformation, and a professional "one-page" dashboard design.

9.2 TECHNICAL ACHIEVEMENTS

The project stands as a verified proof-of-concept for several critical engineering milestones:

- **Decoupled Architecture:** The successful implementation of a RESTful bridge using **Axios** allowed the frontend and backend to scale independently, ensuring that UI updates do not interfere with computational cycles.
- **Data Integrity:** Through the use of **Spring Data JPA**, the project achieved high data reliability, ensuring that user registration and login states are persisted correctly within a relational database.
- **Mathematical Precision:** The custom Java-based quantum engine proved capable of representing complex probability amplitudes and unitary transformations, rendering accurate **Dirac notation** for various 2-qubit states.

9.3 PEDAGOGICAL IMPACT

From an educational perspective, **Q-BIT** effectively lowers the barrier to entry for quantum mechanics. By replacing abstract 4X4 matrices with a visual "Library" of gates and "Wires," the application allows students to "play" with quantum logic. Observing the immediate change in the statevector when a **Hadamard** or **CNOT** gate is applied reinforces the concepts of superposition and entanglement more effectively than static textbook examples.

9.4 CHALLENGES OVERCOME

The journey of building **Q-BIT** was marked by significant technical hurdles, from resolving **MySQL** driver dependencies to fine-tuning the **Lombok** agent in the Eclipse IDE. Overcoming

these challenges not only improved the project's stability but also deepened the developer's expertise in full-stack troubleshooting and system configuration.

9.5 FUTURE SCOPE

While the current version of **Q-BIT** is a robust 2-qubit simulator, there are several avenues for future expansion:

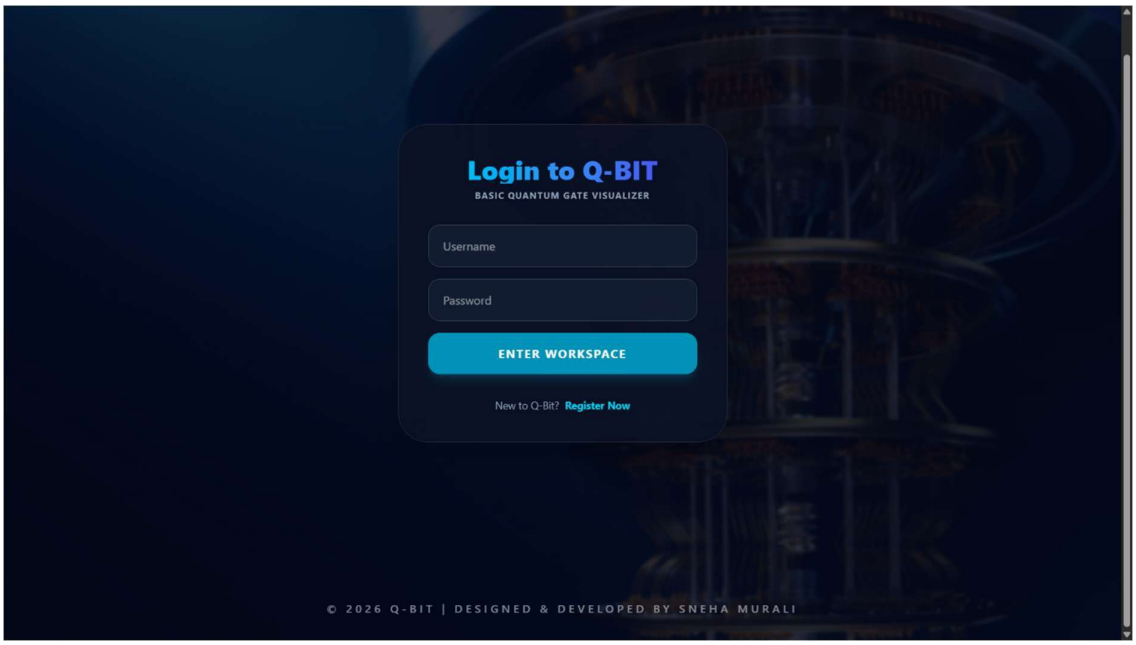
- **Multi-Qubit Scaling:** Future iterations could expand the backend logic to support 3-qubit or 5-qubit systems, enabling the simulation of more complex algorithms like **Grover's Search**.
- **Circuit Exporting:** Implementing a feature to export visual circuits into **OpenQASM** code, allowing users to run their designs on real quantum hardware like IBM Quantum.
- **Advanced Visualizations:** Integrating a 3D **Bloch Sphere** visualizer alongside the wires to provide a geometric interpretation of qubit state rotations.
- **Cloud Deployment:** Migrating the local MySQL and Spring Boot environment to a cloud provider like **AWS** or **Azure** to make the tool accessible to a global audience of students and researchers.

FINAL SUMMARY

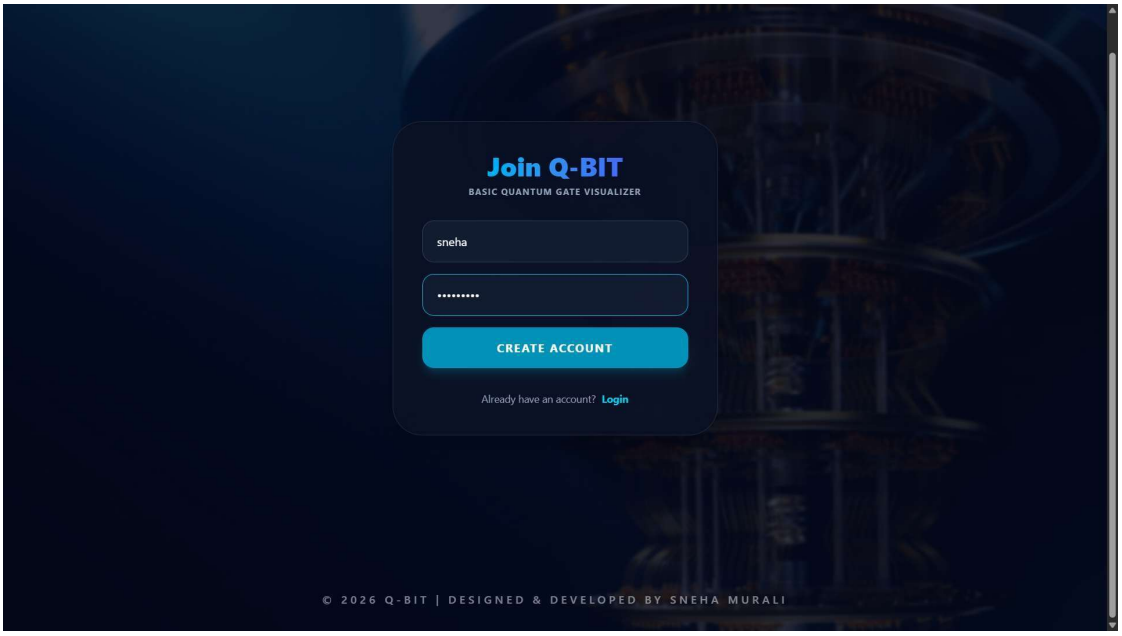
In conclusion, **Q-BIT** is more than a simple visualizer; it is a full-stack engineering solution to a complex educational problem. It marries the precision of Java with the reactivity of React, proving that even the most "quantum" of problems can be solved with a well-architected classical stack.

ANNEXURES

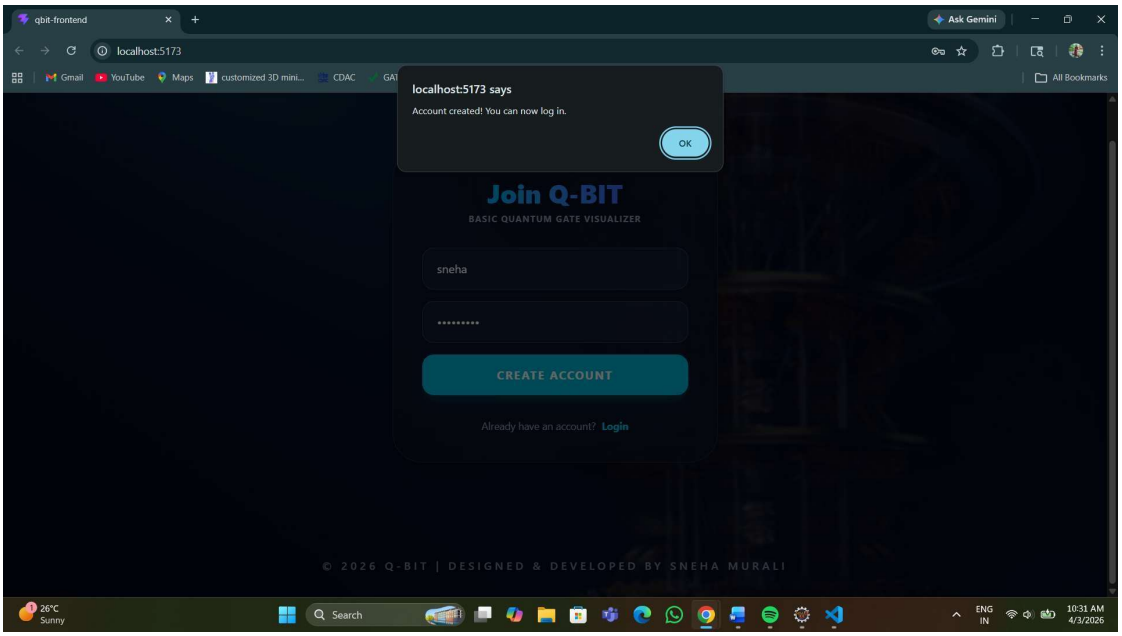
Login Module



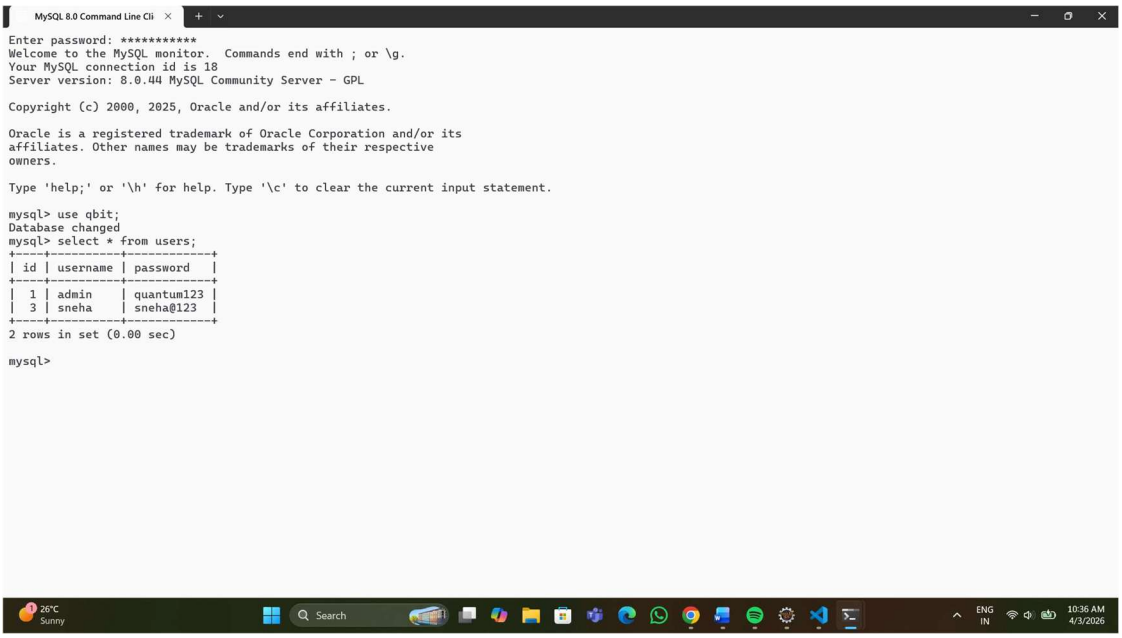
Register Module



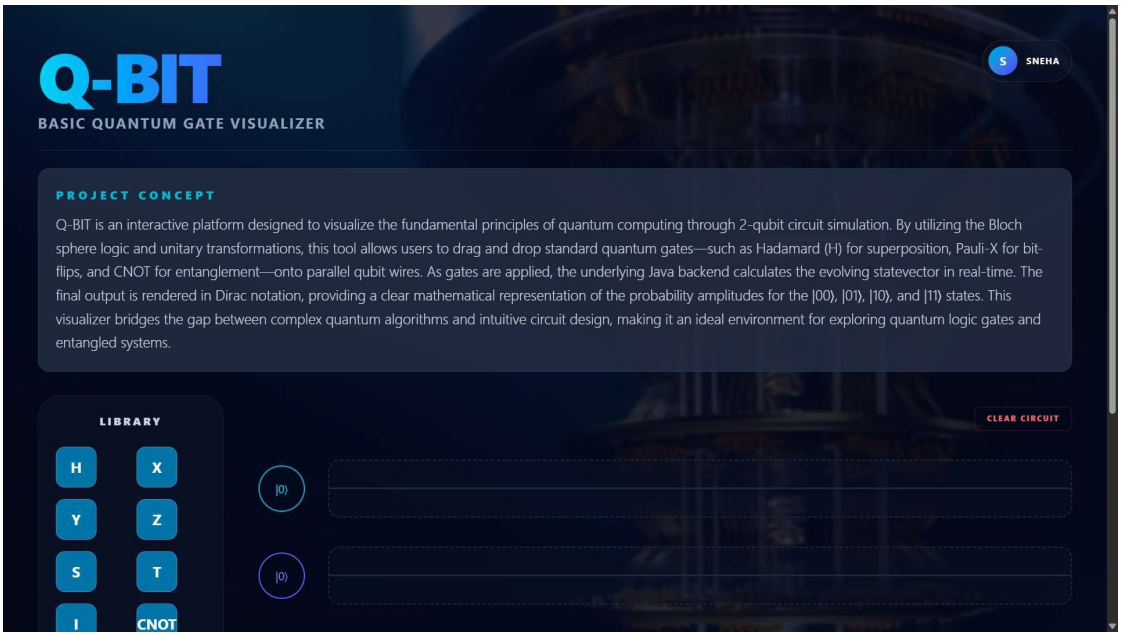
Account Created



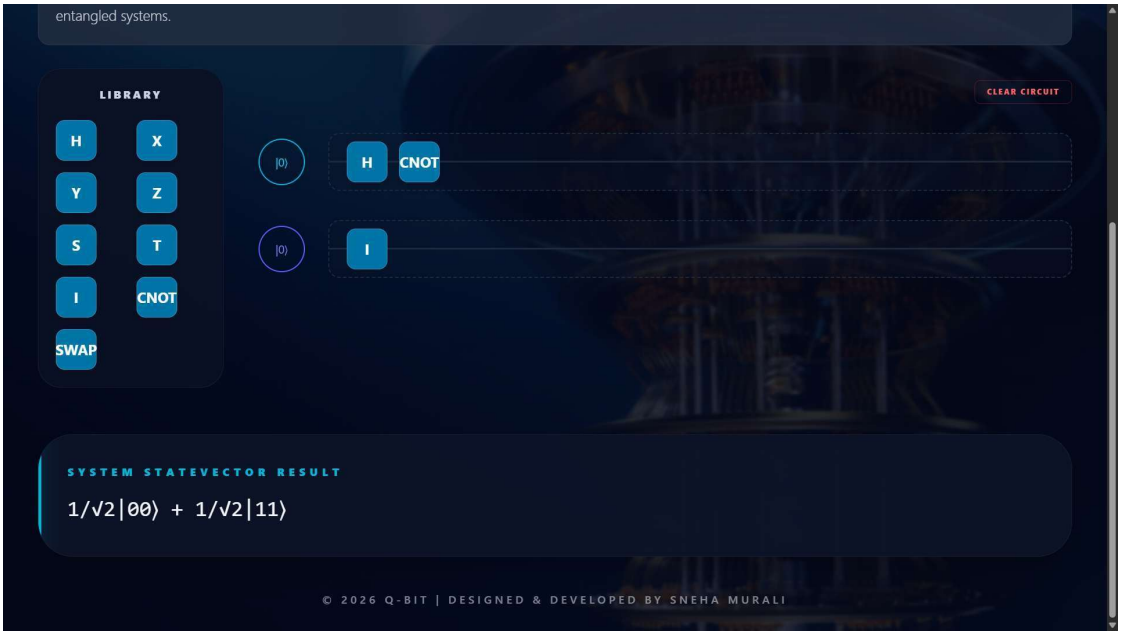
MySQL Database

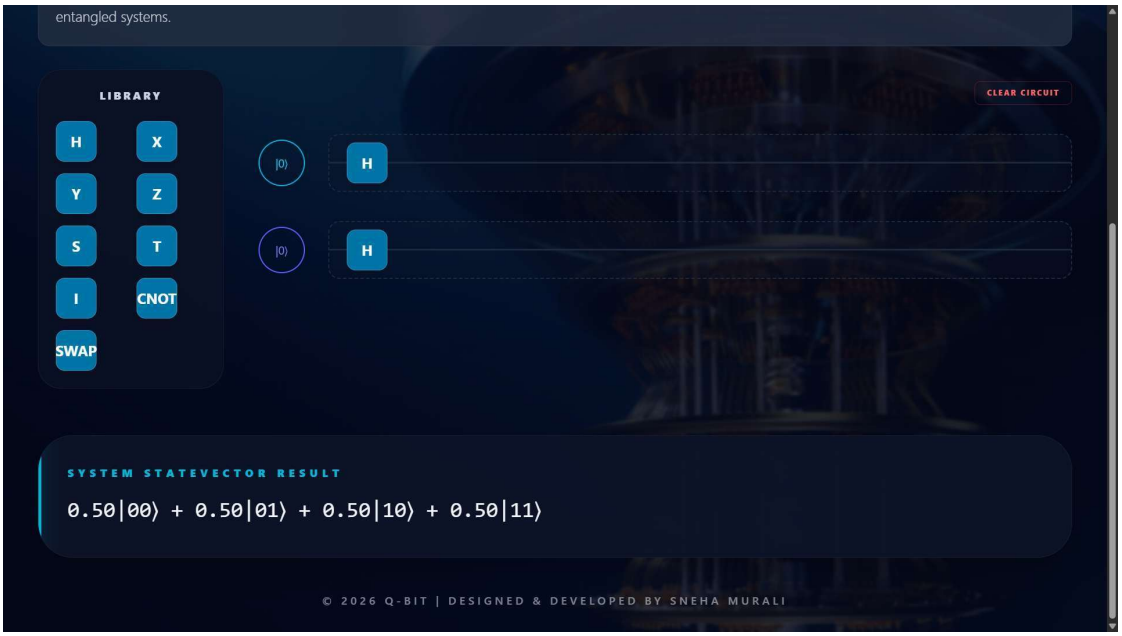


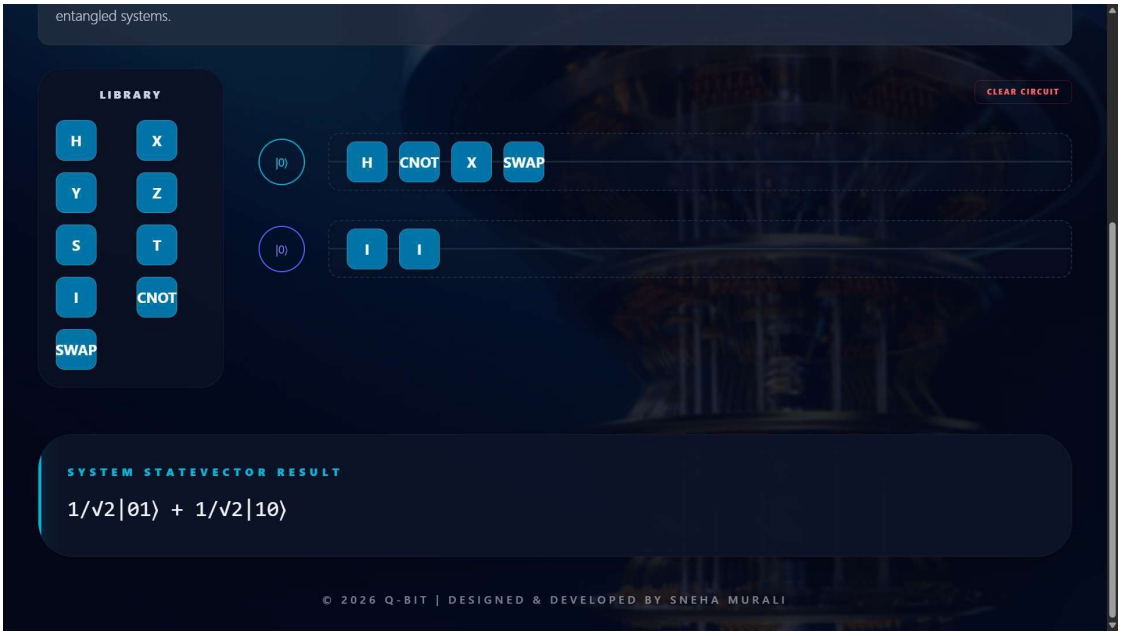
User Interface (Front – end)



Operations




entangled systems.
LIBRARY
H
X
Y
Z
S
T
I
CNOT
SWAP
CLEAR CIRCUIT
|0⟩
H
|0⟩
H
SYSTEM STATEVECTOR RESULT
0.50|00⟩ + 0.50|01⟩ + 0.50|10⟩ + 0.50|11⟩
© 2026 Q-BIT | DESIGNED & DEVELOPED BY SNEHA MURALI


entangled systems.
LIBRARY
H
X
Y
Z
S
T
I
CNOT
SWAP
CLEAR CIRCUIT
|0⟩
H
CNOT
X
SWAP
|0⟩
I
I
SYSTEM STATEVECTOR RESULT
1/√2|01⟩ + 1/√2|10⟩
© 2026 Q-BIT | DESIGNED & DEVELOPED BY SNEHA MURALI